

A Path Planning Method Based on Attention Mechanism and D3QN Algorithm

Jinping Wu

Naval Submarine Academy
Qingdao, China

Jian Liu

Naval Submarine Academy
Qingdao, China

Zichen Liang*

Naval Submarine Academy
Qingdao, China
lag1225@qq.com

Abstract—Path planning is fundamental function for USVs to execute tasks. To address the issue of obstacle avoidance during task execution, this paper proposes a path planning method using an attention mechanism in the D3QN algorithm. The algorithm builds on the existing double deep Q network (DDQN) and the Dueling deep Q network (Dueling DON), introducing a self-attention network to weight the states that the agent should pay attention to. Experimental results from a simulated underwater environment demonstrate that this method improves the algorithm's convergence speed and effectiveness in avoiding obstacles compared to traditional D3QN and DDQN algorithms. This validates the effectiveness of the proposed method and enhances the convergence speed of the algorithm.

Keywords—component; path planning; attention mechanism; D3QN algorithm

I. INTRODUCTION

Unmanned Surface Vehicles (USVs) are crewless surface vessels capable of autonomously executing a variety of tasks, including reconnaissance, surveillance, environmental monitoring, search and rescue, and cargo transportation. By integrating advanced navigation systems, sensors, and communication equipment, USVs can provide effective maritime operation solutions in complex or hazardous environments. Path planning is fundamental to the autonomous navigation capabilities of USVs and is one of the key technologies for executing various tasks. The complexity of the mission environment demands that USVs have the ability to plan routes efficiently and rapidly. The goal of path optimization is to plan the fastest collision-free path from the starting point to the endpoint in a complex environment.

In response to the global path planning problem of USVs in complex waters, Fan Yunsheng [1] and others proposed a genetic algorithm for rapid global path search based on electronic chart gridization. This method extracts marine environmental information from electronic charts, uses grid methods to establish an environmental model for path search space, and encodes path individuals using grid encoding. It employs an improved genetic algorithm with random rapid search to generate an initial population for path search, enhancing the convergence speed and optimization efficiency of global path planning for USVs. Experimental results show that this method is rational and effective. Shu Weinan [2] and others proposed a path planning method based on the improved A*

algorithm, improving the evaluation function to reduce the total number of traversed nodes; smoothing redundant nodes on the planned path to make the path more practical. Zhang Yang[3] and others from the Naval Engineering University proposed an improved artificial potential field algorithm, combined with the performance of the unmanned boat to formulate collision avoidance rules for the unmanned boat, and introduced steering force, improving the APF algorithm to guide the unmanned boat to safely avoid multiple dynamic obstacles in a complex environment while complying with collision avoidance rules. Mao Shouji [4] and others, in order to solve the problems of slow algorithm convergence and falling into local optimal solutions, proposed a bacteria foraging-improved ant colony optimization algorithm. Experimental simulation shows that the global search capability of the improved algorithm has been greatly enhanced, and the number of convergence iterations has been reduced by more than 30%. In response to the problems of multiple motion constraints and slow convergence speed in genetic algorithms for underwater unmanned vehicles, Huang Yuzhou [5] and others proposed an improved genetic algorithm for path planning. This method improves the population initialization process of the genetic algorithm, introduces point grids to improve convergence speed, and combines the idea of catastrophe to avoid the population falling into local optimal solutions. Li Juan [6] and others from Harbin Engineering University proposed a path planning algorithm based on the fusion of DWA and (Rapid-Exploration Random Tree, RRT) algorithms, planning guide points through the improved RRT algorithm, successfully avoiding the problem of DWA falling into a local stagnant state.

The above algorithms have all had certain effects on the path planning of USVs, but for complex environments, traditional heuristic algorithms have phenomena such as slow convergence speed and insufficient computing power. In recent years, the new information technology industry has developed rapidly. With the continuous improvement of computing power, the intelligentization of USVs has become an inevitable trend in the development of USVs at present. Reinforcement learning, as an artificial intelligence solution that requires a lot of computing power support, has achieved good results in the fields of path planning and intelligent decision-making.

II. INTRODUCTION TO ATTENTION MECHANISM IN REINFORCEMENT LEARNING BASICS

A. Reinforcement Learning

Reinforcement Learning (RL) is an important branch of machine learning. It is used to describe and solve the problem of how an agent learns a policy to maximize rewards through interactions with the environment.[8] In reinforcement learning, an agent is an entity capable of observing the environment and taking actions based on those observations. The environment is the external world in which the agent exists and interacts. It can be the physical world or a virtual digital world. A state is a description of the environment at a particular moment. The agent decides what action to take based on the current state. An action is the decision the agent makes in a given state. Actions affect the state of the environment and the rewards the agent receives. Rewards are the feedback the agent gets from the environment after taking an action. They can be positive or negative, guiding the agent to learn which behaviors are good and which are not. A policy is a mapping from states to actions, defining what actions the agent should take in a given state. Figure 1 illustrates the process of interaction between the agent and the environment.

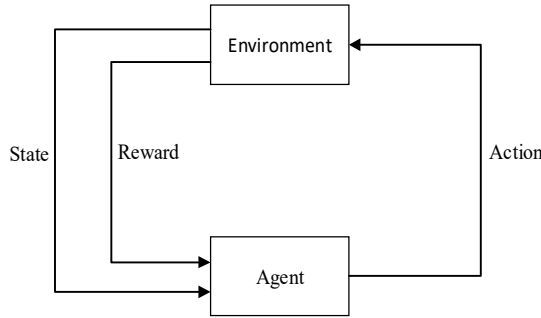


Figure 1. Agent-Environment Interaction Diagram

The core of reinforcement learning is the interaction loop between the agent and the environment. At each time step, the agent observes the state of the environment and then selects an action based on some policy. This action is sent to the environment, which determines the next state and rewards the agent based on its internal rules and the current state. The agent's goal is to learn a policy that maximizes the rewards obtained over the long term.[9] Mainstream reinforcement learning algorithms include DQN, DDPG, TRPO, A3C, and so on.

DQN is a type of deep reinforcement learning algorithm that approximates the Q-function in reinforcement learning using deep neural networks, enabling decision-making in complex environments. In traditional Q-learning, the Q-function is a table that stores the values of each state-action pair, but this method becomes impractical when the state space is large. DQN solves this problem with neural networks, allowing the agent to learn in a high-dimensional state space. At each time step, the neural network takes the current state vector of the agent as input and outputs an n-dimensional Q-value vector, where n is the size of the action space. The agent can then choose the action

corresponding to the maximum Q-value.[12] This algorithm can use experience replay to store past interactions, i.e., tuples of states, actions, rewards, and new states. These experiences are stored in a large buffer (Replay Buffer), from which the agent can randomly sample to train its neural network. This method helps reduce temporal correlations between data, making the learning process more stable.

However, DQN tends to overestimate certain action Q-values during the actual learning process, leading to incorrect learning or decreased efficiency in the agent. To address this issue, the Double DQN (DDQN) algorithm is selected. DDQN employs an asynchronous update method for the Q-network. It first uses the current Q-network to select the action with the maximum Q-value in the next state s_{t+1} , and then evaluates the Q-value of this action using the target Q-network. This helps to reduce estimation bias. The Q-value update logic in the DQN network is as follows:

$$Q(s_t, a_t) \leftarrow r_t + \max_a Q(s_{t+1}, a_{t+1}) \quad (1)$$

The value update logic in the DDQN network:

$$Q(s_t, a_t) \leftarrow r_t + Q_target(s_{t+1}, \arg \max_a Q(s_{t+1}, a_{t+1})) \quad (2)$$

In the DDQN network, r_t represents the immediate reward the agent receives at time step t in the current state, and $\max_a Q(s, a)$ refers to the maximum Q-value among all possible actions in that state s .

Additionally, to enhance learning efficiency, the Dueling DQN algorithm can be combined. This algorithm decomposes the Q-value into a state value function $V(s)$ and an advantage function $A(s, a)$. The state value function estimates the expected return of being in a particular state, regardless of the action taken, while the advantage function estimates the relative benefit of taking a particular action in a given state compared to other actions.[11] This separation allows the network to learn the value of states and the relative advantages of actions more effectively.

$$Q(s_t, a_t) = V(s_t) + A(s_t, a_t) \quad (3)$$

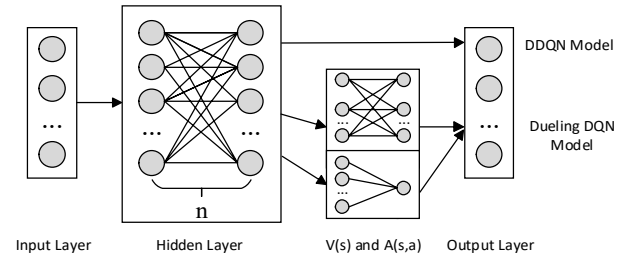


Figure 2. Structure diagram of DDQN and Dueling DQN networks

Traditional DQN algorithms only update the Q-value for a specific action, and the Q-values for other actions are not updated. In the Dueling DQN algorithm, the $V(s)$ function is updated at every time step, which also affects the Q-values for other actions.

Therefore, Dueling DQN can learn the state value function more frequently and accurately. Figure 2 illustrates the network architectures of Double DQN and Dueling DQN.

The combination of DDQN and Dueling DQN algorithms is called the D3QN algorithm.[10]

B. Self-Attention Mechanism

The attention mechanism is a widely used model in deep learning, especially when dealing with sequential data, such as in natural language processing (NLP) and computer vision (CV).[7] Its core idea is to enable the model to focus on the most important parts of the input data, thereby improving the model's performance and interpretability.

The working principle of the attention mechanism can be likened to human attention. When humans read or observe, they well concentrate on the most relevant information and ignore other unimportant parts. In machine learning, the attention mechanism achieves this by calculating the weights of each element in the input sequence. These weights are usually generated by a learnable function that considers the current contextual information and the sequence elements themselves.

In traditional neural networks, such as recurrent neural networks (RNN) or convolutional neural networks (CNN), information is passed through the hidden layers at each step of the sequence. However, these networks encounter problems with vanishing or exploding gradients when dealing with long sequences, making it difficult to learn long-term dependencies. The attention mechanism effectively solves this problem by allowing the model to establish dynamic weighted connections between different parts of the sequence.

In reinforcement learning, self-attention mechanisms can help the agent focus on the most important parts of the state, thereby improving the agent's learning efficiency. This article introduces a self-attention network into the aforementioned D3QN network, composed of Query (Q), Keys (K), and Values (V) as the inputs of the attention mechanism network. Q, K, and V are usually composed of matrices. In reinforcement learning, the state vector state is one-dimensional, so Q, K, and V here are all one-dimensional vectors.

$$\begin{cases} Q = W_Q \cdot state + b_Q \\ K = W_K \cdot state + b_K \\ V = W_V \cdot state + b_V \end{cases} \quad (4)$$

Here, W_Q, W_K, W_V are learnable weight matrices, and b_Q, b_K, b_V are the corresponding bias terms, which are optimized through back propagation during the training process.

In the attention mechanism network, the first step is to calculate the attention scores.

$$scores = \frac{QK^T}{\sqrt{d_k}} \quad (5)$$

Here, a mask sequence M is introduced, which serves to either increase or decrease the scores of certain elements when calculating attention scores in the network.

$$M = [m_1, m_2, \dots, m_n] \quad (6)$$

$$Masked\ Scores = scores + M \quad (7)$$

The size of m determines which elements the model should focus on, and it is a parameter between 0 and 1. Then, the *softmax* function is applied to normalize the attention scores so that the sum of the attention weights equals 1.

$$weight_i = \frac{\exp(scores_i)}{\sum_{j=1}^n \exp(scores_j)} \quad (8)$$

$$Weights = softmax(Masked\ Scores) \quad (9)$$

Finally, the weights are used to perform a weighted sum on V , resulting in the output that has been processed by the attention mechanism.

C. SA-D3QN Structure

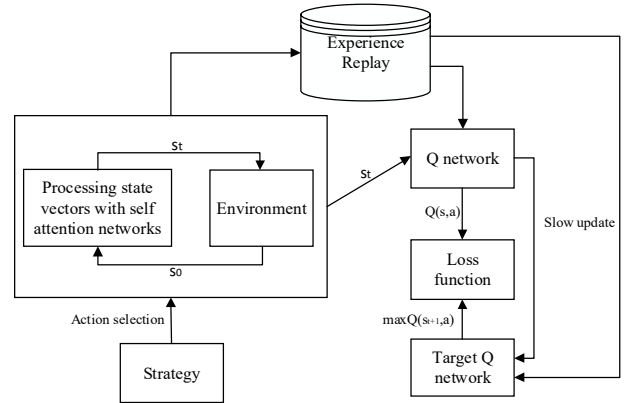


Figure 3. SA-D3QN Algorithm Framework

In this algorithm, a combination of deep neural networks and self-attention networks is utilized. By stacking self-attention mechanism networks, the model can better focus on the more important environmental parameters within the state vector, thereby enhancing the agent's ability to cope with complex underwater environments. In the implementation of the programming, both our evaluation network and target network incorporate a structure that combines three layers of self-attention layers with deep neural networks. In the self-attention network, the input to each layer is the output of the previous layer. Once the model receives environmental information, it uses the self-attention network to enhance the features of the state vector. Then, the agent selects actions based on the enhanced state vector through an ϵ -greedy strategy. Subsequently, the actions and the enhanced state vectors are transmitted to the neural network to calculate the Q-values and rewards, which are used to evaluate the quality of the actions. Finally, the model is trained and

updated through experience replay mechanisms by randomly sampling experiences. The algorithm framework is shown in Figure 3, and the algorithm flow is shown in Table 1.

TABLE I. ATTENTION MECHANISM-BASED D3QN REINFORCEMENT LEARNING PATH PLANNING ALGORITHM FLOW

Input: Capacity of experience replay pool M , number of training episodes N , greedy factor epsilon, discount factor gamma, target Q-network parameter update step size C . Output: Trained neural network.
for episode = 1 to M do
Obtain the initial state vector s_0 from the environment.
Process s_0 through the self-attention network to get the enhanced state vector.
for step = 1 to N do
Choose action a_t according to the epsilon-greedy strategy.
Execute action a_t , update state s_{t+1} , and obtain reward r_t .
Store the transition (s, a, r, s') in the experience replay pool and update.
Sample data from the experience replay pool for training.
Perform gradient descent on the parameters theta using the loss function gradient $Loss = E[(\gamma_j - Q(s_j, a; \theta))^2]$
Update the target Q-network every C steps.
end for
end for

III. ATTENTION MECHANISM-BASED REINFORCEMENT LEARNING PATH PLANNING METHOD

A. Environmental Model

In the construction of the environment, the design of obstacles is the first step. In actual situations, obstacles mainly include: reefs, shoals, and mines laid during wartime, etc. To reflect the complexity of real-world scenarios, a total of seven obstacles are established in the environment. Each obstacle generates its top-left coordinates (x_A, y_A) and top-right coordinates (x_B, y_B) , and a line segment extending from these two points can construct a square obstacle of any size. Theoretically, the random combination of multiple square obstacles can simulate models of almost any actual obstacle in the experimental environment. Due to the random generation of obstacles, this method can simulate any maritime environment, possessing universality.

When USV perform tasks underwater, they primarily rely on sonar to detect and identify obstacles in their surrounding environment. The sonar systems carried by USV can generally cover all directions around them, capable of detecting obstacles within a certain distance in a 360° radius around the USV. To simplify the state space, speed up computations, and prevent gradient explosion, this article have simplified the sonar to twelve rays spaced every 30° around the USV. When a ray comes into contact with an obstacle, it returns the distance d_i from the obstacle to the USV along that ray. If a ray does not

come into contact with an obstacle, it returns a default distance of infinity. There are $\{d_1, d_2, \dots, d_{12}\}$ distances in total.

To enable the agent to interact and learn better with the environment, it is necessary to constantly pay attention to the relative position of the agent with respect to the destination during the process of moving towards the destination. In the path planning process of USV, the agent knows its own position (x_0, y_0) and the position of the destination (x_1, y_1) , so it can calculate the Euclidean distance L from the agent to the destination in real time. s

$$L = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2} \quad (10)$$

Additionally, to enhance the agent's planning capabilities, the current movement direction α of the USV and the relative angle β between the USV and the destination are calculated in the environment, and the angle θ between these two angles is determined.

$$\theta = \alpha - \beta \quad (11)$$

Combining the above parameters forms a 14-dimensional state space. $\{L, d_1, d_2, d_3, d_4, d_5, d_6, d_7, d_8, d_9, d_{10}, d_{11}, d_{12}, \theta\}$

B. Action Model

For general small USV, their self-contained power supply and portability limit their maneuverability. Considering that in actual situations, the turning angle of a USV should not be too large at one time, the turning angle of the USV is limited to 20° degrees here. To make the action space more reasonable and to avoid having too many dimensions in the action space, the avoidance behavior of the USV is simplified into five actions: turning left once or twice, turning right once or twice, and not turning. Compared to an omnidirectional 360° or four actions of forward, backward, left, and right, this action space is more reasonable and has fewer dimensions. The agent interacts with the environment at time step t and selects its action based on the action space.

In the process of the agent selecting actions, appropriate exploration of new actions is very necessary. Too little exploration can lead the agent's learned strategy into a local optimal solution, while too much exploration can slow down the convergence speed. To balance this situation, this paper chooses the epsilon-greedy strategy, which regulates the probability of the agent exploring new actions by designing the size of epsilon. Epsilon is a parameter between 0 and 1; the larger the epsilon, the greater the probability that the agent will randomly choose actions, and the smaller the epsilon, the more the agent will choose based on the learned strategy. At the same time, to improve the convergence speed, this paper dynamically adjusts the value of epsilon, designing a decay rate that makes epsilon gradually decrease as the number of training episodes increases. That is, more exploration of new actions is conducted in the early stages of training to avoid falling into local optima.

C. Reward Model

The reward function can guide the agent to choose the appropriate action in different states. The rationality of the reward function is directly related to whether the agent can correctly learn the required strategy. The learning objective of the agent is to maximize the expected total reward.

$$G_t = \sum_{i=0}^{\infty} \gamma^i r_i \quad (12)$$

G_t is the total reward, γ is the discount factor, a parameter between 0 and 1, reflecting the decay effect of rewards at future times. r_i is the reward function, which is the immediate reward obtained when the agent takes action a in state s and transitions to state s_{t+1} . When setting rewards, they can generally be divided into delayed rewards and immediate rewards. In some cases, the agent needs to wait for multiple time steps to receive rewards for its actions, which are called delayed rewards. Delayed rewards may slow down the learning process because the agent has difficulty associating long-term actions with the final reward outcomes. This requires immediate rewards to guide the agent. Immediate rewards refer to the feedback the agent receives immediately after taking an action. This form of reward emphasizes causality, allowing the agent to quickly understand the direct consequences of its actions.

In this path planning task, the main goal of the agent is to navigate to the end point while avoiding obstacles. Therefore, a reward should be given when the agent reaches the end point, and a penalty should be deducted when the agent hits an obstacle. To prevent the program from running indefinitely, a truncation operation is needed. A penalty should also be deducted when the agent reaches a certain number of steps. All of the above are delayed rewards. To guide the agent's learning, the design of immediate rewards includes: at each time step, a penalty should be deducted if the agent gets too close to an obstacle, a reward should be given if the agent moves towards the end point, and a reward should be given if the agent is closer to the end point than in the previous state. The specific details are shown in Table 2.

TABLE II. REWARD DESIGN

Category of awards	Value
Arrive at the end point	+20
Hit an obstacle	-20
Too many steps(>500)	-10
Move towards the end point	+1
Too close to an obstacle	-1
Close to the end point	+0.5
Far from the end point	-1

IV. EXPERIMENTAL SIMULATION AND RESULT ANALYSIS

A. Training Parameter Settings and Software/Hardware Condition

The hardware for this paper's simulation experiment includes: GPU Nvidia GTX 3070Ti, CPU AMD Ryzen 9 6900HX, and 32GB of memory; the software includes: operating system Windows 10, Python 3.12 for experimental simulation, with main modules such as pytorch, matplotlib, and gym, etc. The optimizer used is the commonly used Adam optimizer. Hyperparameter settings are shown in Table 3.

TABLE III. ALGORITHM HYPERPARAMETER SETTINGS

Hyper parameter	Value
Discount factor	0.99
Training episodes	2000
Learning rate	1e-2
Sample batch size	128
Experience replay pool capacity	100000000
Epsilon-greedy factor	0.2

B. Simulation Results

In the simulation experimental environment, the same training parameter settings were used for the SA-D3QN algorithm, D3QN algorithm, and DDQN algorithm for simulation experiments, and the experimental results were obtained.

Figure 4 and Figure 5 show the moving average reward comparison between the SA-D3QN algorithm and the D3QN algorithm and DDQN algorithm, respectively. Each point on the graph records the average of the rewards for the previous 100 episodes. By comparison, it can be seen that the SA-D3QN algorithm for path planning has a faster convergence speed of model rewards.

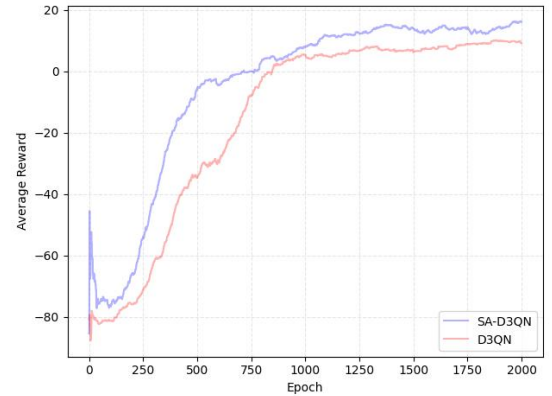


Figure 4. SA-D3QN vs. D3QN Moving Average Reward Comparison

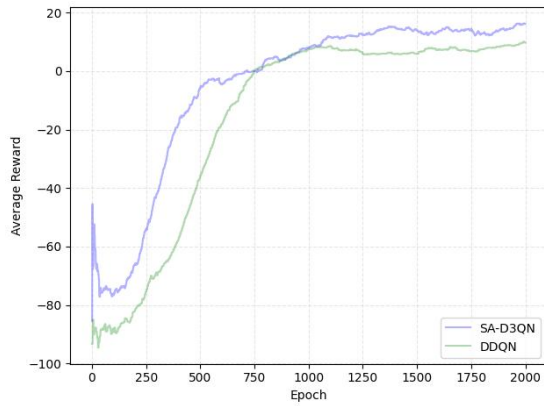


Figure 5. SA-D3QN vs. DDQN Moving Average Reward Comparison

Figure 6 shows the simulation experimental results after training the SA-D3QN algorithm 2000 times, where the agent finds a path.

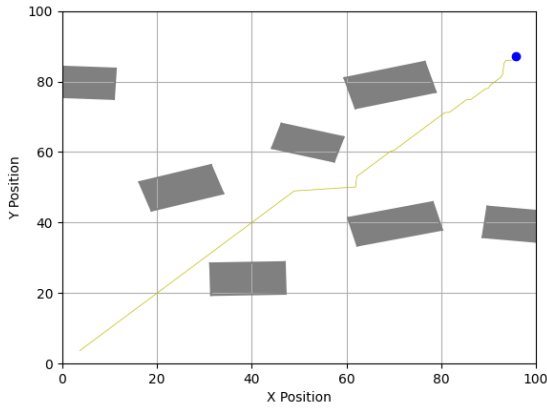


Figure 6. SA-D3QN Algorithm Training Path

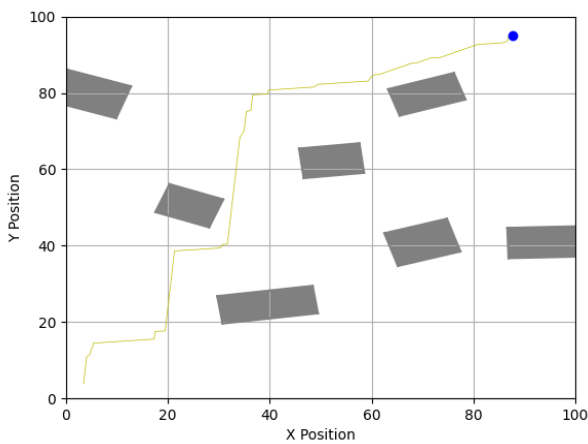


Figure 7. D3QN Algorithm Training Path

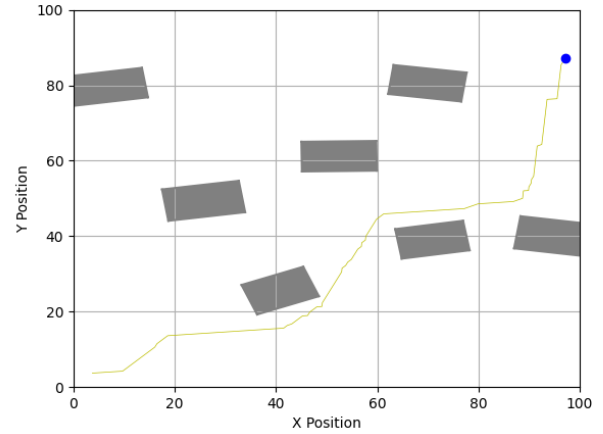


Figure 8. DDQN Algorithm Training Path

Figures 7 and 8 show the simulation experimental results after training the D3QN algorithm 2000 times without the attention mechanism, where the agent also finds an avoidance path.

Following that, after training the agents, we respectively counted the number of inflection points generated when the agents successfully reached the end point, that is, the number of times the USV turned. The number of inflection points can well reflect the smoothness of the path. Table 4 shows the results of 100 path planning statistics for agents trained with the SA-D3QN algorithm, D3QN algorithm, and DDQN algorithm, respectively.

TABLE IV. NUMBER OF INFLECTION POINTS

Category	Average	Minimum
<i>SA-D3QN</i>	16	10
<i>D3QN</i>	24	12
<i>DDQN</i>	27	12

From the table above, it can be observed that the SA-D3QN algorithm produces the fewest inflection points in the path after training, indicating a smoother path. Although the minimum number of inflection points for the D3QN and DDQN algorithms is not significantly different from that of the SA-D3QN, the average value of the SA-D3QN is smaller. This demonstrates that the SA-D3QN algorithm has better performance and stability, with a higher frequency of producing smoother paths.

V. CONCLUSIONS(HEADING 5)

Path planning is the most fundamental function for USVs in performing various tasks. This paper innovates on the traditional D3QN algorithm by introducing a self-attention network to weight the state vectors. It then models the complex maritime environment, USV state space, action space, and reward function. Through experimental simulation, this article compares the moving averages of the SA-D3QN algorithm and the D3QN algorithm with the DDQN algorithm, as well as the average and minimum inflection points of the USV path planning routes trained by different algorithms. The results show that SA-D3QN

method improves the convergence speed and effectiveness of the USV's obstacle avoidance path planning algorithm, thereby validating its effectiveness.

REFERENCES

- [1] Fan Yunsheng, Zhao Yongsheng, Shi Linlong, et al. Global path planning for unmanned surface vehicles based on grid-based electronic charts [J]. *China Navigation*, 2017, 40(01): 47-52+113.
- [2] Shu Weinan, Zhao Jiansen, Xie Zongxuan, et al. Path planning for unmanned surface vehicles based on improved A* algorithm [J]. *Journal of Shanghai Maritime University*, 2022, 43(02): 1-6.
- [3] Zhang Yang, Peng Pengfei, Cao Jie. Local path planning for unmanned surface vehicles based on improved APF algorithm [J]. *Journal of Ordnance Equipment Engineering*, 2023, 44(09): 42-48.
- [4] Mao Shouqi, Yang Ping, Gao Dijun, et al. Path planning for unmanned surface vehicles based on bacterial foraging-improved ant colony optimization algorithm [J]. *Control Engineering*, 2024, 31(04): 608-616.
- [5] Huang Yuzhou, Hu Qingyu, Xiong Huaqiao. Path planning for an unmanned underwater vehicle based on improved genetic algorithm. *Digital Ocean and Underwater Defense*, 2024, 7(2): 215-224.
- [6] Li Juan, Zhang Zihao, Zhang Honghan. Local path planning for AUV in complex environments using a DWA and RRT algorithm integrate approach. *Intelligent Systems Journal*.
- [7] Gong Zheng, Feng Yong. One-to-many charging scheduling method for WRSN based on attention mechanism and value reinforcement learning. *Sensor Technique Journal*, 2024, 37(08): 1411-1423.
- [8] Wang Jianming, Wang Xin, Li Yanghui, et al. Path planning for parking robots based on improved D3QN algorithm. *Computers and Moderization*, 2024, (03): 7-14.
- [9] Guo Mengzhu, Sun Jun. Congestion control algorithm based on reinforcement learning D3QN, *Computer Techniques and Development*, 2023, 33(02): 105-109+118.
- [10] Yuan Shuai, Zhang Lili, Gu Qiran, et al. Study on mobile robot path planning method based on priority sampling D3QN, *Journal of Small and Micro Computer System*, 2023, 44(05): 923-929.
- [11] Zhao Xinyi. Study on path planning method for UUV based on curriculum-style dual DQN, Harbin Engineering University, 2021.
- [12] Huang Y, Wei G, Wang Y. V-D D3QN: A variant of the deep dueling Q-learning network with a dueling architecture. *Proceedings of the 37th Chinese Control Conference*, F, School of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology, 2018: 6.